

PS159 SecureMailScope MVP Workflow

2-Person, 2-Day Build Workflow — From Zero to Demo

Team SecureMailScope

SIH National Screening — Working Document

0. How to Use This Document
1. Roles and the One Contract Between You
2. Dependency Map
3. Tools and Environment
4. GitHub Workflow
5. Day 1
6. Day 2
7. Fallback Rules — Apply Automatically, Do Not Debate Under Time Pressure
8. Demo Script (2-3 Minutes)
9. Judge Q&A — Rehearse These Answers
10. Definition of Done

0. How to Use This Document

This is written the way a senior security engineer would hand off a red-team-informed build plan to two developers under a hard deadline: assume nothing, state every dependency explicitly, and make every failure mode a planned fallback instead of a surprise at 11 PM.

Two people. Two days. One interface between you: a JSON file. Everything below is organized so you can each read only your own lane and never need to touch the other's code.

The attacker's-eye view, briefly, because it shapes what you're actually grading: everything this tool flags is a real technique someone would use to read email they shouldn't. Old TLS versions and weak ciphers (RC4, DES, export-grade, anything MD5-signed) are broken enough that a patient attacker with recorded traffic can decrypt it later. Missing forward secrecy means a single stolen private key unlocks every past session, not just future ones. STARTTLS stripping is an active man-in-the-middle move — an attacker on the network path deletes the STARTTLS offer from the server's reply, and a badly-written client just... continues in plaintext, no error, no warning, credentials and content both exposed. Expired or self-signed-where-it-shouldn't-be certificates are what let an attacker's server impersonate the real one. You are not building a checklist tool. You are building the thing that catches the mail server that *looks* fine until someone actually tries these tricks against it. Keep that framing in your head — it's also exactly what a judge will probe you on.

1. Roles and the One Contract Between You

Person A — Analysis Engine. Owns everything from raw PCAP to a structured JSON result. Never touches the dashboard.

Person B — Dashboard. Owns everything from a JSON result to what a human sees on screen. Never touches tshark, packet parsing, or certificate internals.

The contract — `analysis_result.json`. This is the *only* thing that crosses between you. Agree on it in the first 30 minutes of Day 1 and do not change its shape casually afterward — if a field needs to change, that’s a five-minute conversation between you, not a silent edit.

```
{
  "metadata": {
    "filename": "weak.pcap",
    "analyzed_at": "2026-09-27T10:00:00Z"
  },
  "summary": {
    "total_sessions": 3,
    "low_risk": 1,
    "medium_risk": 0,
    "high_risk": 2
  },
  "sessions": [
    {
      "session_id": "session_001",
      "protocol": "SMTP",
      "tls": {
        "detected": true,
        "version": "TLS 1.2",
        "cipher_suite": "TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256",
        "key_exchange": "ECDHE"
      },
      "certificate": {
        "present": true,
        "subject": "CN=mail.example.com",
        "issuer": "CN=Test CA",
        "valid_from": "2025-01-01",
        "valid_until": "2026-01-01",
        "expired": false
      },
      "starttls": {
        "attempted": true,
        "succeeded": true,
        "note": null
      },
      "findings": [
        {
          "title": "Modern TLS and valid certificate",
          "severity": "LOW",
          "description": "Session uses TLS 1.2 with a strong cipher suite and a valid certificate.",
          "evidence": "tcp.stream==4",
          "recommendation": "No action needed."
        }
      ],
      "risk": { "level": "LOW", "score": 10 }
    }
  ]
}
```

Non-negotiable rules for this file:

- A field that can’t be reliably extracted is `null`. Never invented. A dashboard that says “unknown” survives judge questions; a dashboard that shows a fabricated cipher suite does not.
- Person B never opens Person A’s Python. Person A never opens Person B’s dashboard code. If either of

you needs to touch the other's code to make something work, the schema is wrong — fix the schema, not the wall between you.

2. Dependency Map

This is the entire dependency graph of the project. Everything not listed here can be built in parallel.

Task	Owner	Depends on
JSON schema agreement	A + B	Nothing — do this first
Test PCAP sourcing	A	Nothing
tshark session identification	A	Test PCAPs
TLS handshake extraction	A	Session identification
Certificate extraction	A	TLS handshake extraction
STARTTLS detection (stretch)	A	Session identification
Security rule grading	A	TLS + cert extraction
Mock JSON files	B	Schema agreement
Dashboard skeleton	B	Schema agreement
Dashboard fully working on mocks	B	Mock JSON
Real JSON output (good.json , weak.json)	A	Full analyzer pipeline
Dashboard swapped to real JSON	B	A's real JSON output
Live PCAP-upload integration	B	A's analyzer.py callable as CLI/subprocess
Final demo rehearsal	A + B	Everything above

Read this table as permission to work independently. Only four things are true blockers: the schema (Day 1, first 30 minutes), Person A's real JSON existing at all (end of Day 1), Person A's *finished* JSON (Day 2), and the final integration test (Day 2 evening). Everything else, each of you owns alone.

3. Tools and Environment

Tool	Used by	Purpose	Install
tshark	A	Packet/protocol/TLS decoding — do not hand-parse packets, tshark's dissectors are mature and battle-tested	<code>sudo apt install tshark</code> <code>/ brew install wireshark</code>
Python 3.10+	A + B	Everything	Usually preinstalled; <code>python3 --version</code> to check
<code>cryptography</code> (pip)	A	Certificate parsing/validation	<code>pip install cryptography</code>
Streamlit	B	Dashboard — fastest path to a working UI for one person in one day	<code>pip install streamlit</code>
pandas	B	Table rendering in the dashboard	<code>pip install pandas</code>
Git + GitHub	A + B	Version control, parallel work without stepping on each other	<code>git --version</code> to check
Wireshark sample captures	A	Free real-world SMTP/IMAP/TLS test traffic	web search “Wireshark SampleCaptures”
Netresec public PCAP repo	A	Broader real-world traffic variety, including weak/old TLS	web search “Netresec public PCAP repository”

Setup commands, run once each, Day 1 morning:

```
# tshark – verify field names for your installed version before relying
# on any tshark command below; field names shift slightly across versions
tshark -v
tshark -G fields | grep -i x509

python3 -m venv venv
source venv/bin/activate
pip install streamlit pandas cryptography
```

requirements.txt :

```
streamlit
pandas
cryptography
```

4. GitHub Workflow

Two people, two days — you do not need a heavyweight branching model. You need to never block each other and never have a merge conflict eat an hour you don't have.

Repository structure:

```
PS159-MVP/
├── analyzer/
│   ├── pcap_parser.py
│   ├── tls_analyzer.py
│   ├── certificate_analyzer.py
│   ├── starttls_analyzer.py
│   ├── security_rules.py
│   └── analyzer.py
├── dashboard/
│   └── app.py
├── data/
│   ├── good.pcap
│   ├── weak.pcap
│   └── critical.pcap
├── sample_results/
│   ├── good.json
│   ├── weak.json
│   └── critical.json
├── requirements.txt
└── README.md
```

Branch strategy — deliberately simple:

- `main` — always the last known-working state. Never push straight to it after Day 1 morning.
- `analyzer` — Person A's branch. Person A works here all of Day 1 and Day 2, commits freely, never worries about breaking the dashboard because nothing here touches it.
- `dashboard` — Person B's branch, same deal.

Because the only shared surface is JSON files under `sample_results/`, not code, merge conflicts are structurally almost impossible. This is the actual point of the schema-first design: Git enforces the same separation the architecture already gives you.

Sync points (the only times you touch `main`):

1. **Day 1, end of day.** Person A merges `analyzer` → `main` once `good.json` exists and matches the schema. Person B merges `dashboard` → `main` once the mock-data dashboard is fully working. Test together: does `main`'s dashboard render `main`'s real JSON with zero code changes? If yes, you're on schedule.
2. **Day 2, midday.** Person A merges updated analyzer output (`weak.json`, `critical.json`, `STARTTLS` field if working). Quick re-check against the dashboard.
3. **Day 2, evening.** Final merge, final integration test, tag it:

```
git tag -a v1.0-demo -m "SIH MVP demo build"
git push origin v1.0-demo
```

Commit discipline (keeps you both sane, costs nothing):

```
git commit -m "analyzer: extract TLS version and cipher suite from ServerHello"
git commit -m "dashboard: add risk-level color coding to session table"
```

Prefix with your area, one logical change per commit. If your demo breaks at 11 PM, you want `git log` to actually tell you what changed.

One rule that matters more than the branch model: never rewrite `sample_results/*.json` by hand once Person A's analyzer is producing them for real. If the dashboard shows something wrong, that's a bug to find, not a value to quietly edit — a hand-edited “correct” JSON is exactly the kind of thing that falls apart the moment a judge asks you to run it live.

5. Day 1

5.1 Both — First 30 Minutes

Lock the JSON schema in Section 1. Adjust field names only after Person A has actually run tshark once and confirmed what's realistically extractable — don't design the schema in the abstract, design it against what the tool can actually give you.

5.2 Person A — Day 1

Step 1 — Source test PCAPs (30-60 min). Do not build your own mail server — that's a half-day sink you don't have. Pull real samples: search "Wireshark SampleCaptures SMTP" and "Netresec public PCAP repository" for SMTP/IMAP sessions, ideally one clean modern-TLS example and one weak/old-TLS or plaintext example. Rename them `good.pcap` and `weak.pcap`. If nothing perfect turns up in 20 minutes, use whatever real SMTP/TLS pcap you can find — proving the pipeline works end-to-end matters more than a perfect dataset.

Step 2 — Identify sessions (30-60 min).

```
tshark -r data/good.pcap -Y "tcp.port==25 || tcp.port==587 || tcp.port==143 || tcp.port==993 || tcp.port==110 || tcp.port==995" \
-T fields -e tcp.stream -e tcp.port -e ip.src -e ip.dst -E header=y > sessions.csv
```

Map port to protocol in Python (25/587 → SMTP, 143/993 → IMAP, 110/995 → POP3). Each unique `tcp.stream` value is one session.

Step 3 — TLS handshake fields (1-2 hrs).

```
tshark -r data/good.pcap -Y "tls.handshake.type==1 || tls.handshake.type==2" \
-T fields -e tcp.stream -e tls.handshake.type -e tls.handshake.version -e tls.handshake.ciphersuite \
-E header=y -E separator=, > tls_handshakes.csv
```

`tls.handshake.type==1` is ClientHello, `==2` is ServerHello. The ServerHello row carries the *negotiated* version and cipher — that's what you record.

Step 4 — Certificate fields (1 hr).

```
tshark -r data/good.pcap -Y "tls.handshake.certificate" \
-T fields -e tcp.stream -e x509af.notBefore -e x509af.notAfter -e x509if.printableString \
-E header=y -E separator=, > certs.csv
```

These field names are the most likely to drift between tshark versions. Confirm with the `tshark -G fields | grep -i x509` check from Section 3 first. If they don't match, `tshark -r file.pcap -Y "tls.handshake.certificate" -V` dumps the fully decoded structure so you can find the correct field names by hand.

Step 5 — Security rules skeleton (`security_rules.py`):

```
def grade_tls(tls):
    if not tls or not tls.get("detected"):
        return "HIGH", "No TLS detected – session in plaintext"
    if tls["version"] in ["SSLv3", "TLS 1.0", "TLS 1.1"]:
        return "HIGH", f"Deprecated TLS version: {tls['version']}"
    weak_markers = ["RC4", "DES", "EXPORT", "NULL", "MD5"]
    if tls.get("cipher_suite") and any(w in tls["cipher_suite"] for w in weak_markers):
        return "HIGH", f"Weak cipher suite: {tls['cipher_suite']}"
```

```

        return "LOW", "Modern TLS version and cipher"

def grade_cert(cert):
    if not cert or not cert.get("present"):
        return "MEDIUM", "No certificate found in handshake"
    if cert.get("expired"):
        return "HIGH", "Certificate expired"
    return "LOW", "Certificate present and valid"

```

End-of-day milestone: analyzer.py runs on good.pcap and produces one real analysis_result.json matching the schema, even with some fields null. This is the single most important checkpoint of Day 1 — if it slips, Day 2 planning changes.

5.3 Person B — Day 1

Step 1 — Write 3 mock JSON files matching the schema exactly. Save as sample_results/mock_good.json, mock_weak.json, mock_critical.json.

Step 2 — Dashboard skeleton (dashboard/app.py):

```

import streamlit as st
import pandas as pd
import json

st.set_page_config(page_title="SecureMailScope", layout="wide")

@st.cache_data
def load_data(path):
    with open(path) as f:
        return json.load(f)

st.sidebar.title("SecureMailScope")
choice = st.sidebar.selectbox(
    "Load capture",
    ["mock_good.json", "mock_weak.json", "mock_critical.json"]
)
data = load_data(f"sample_results/{choice}")

st.title("Email Traffic Security Dashboard")

col1, col2, col3, col4 = st.columns(4)
col1.metric("Total Sessions", data["summary"]["total_sessions"])
col2.metric("Low Risk", data["summary"]["low_risk"])
col3.metric("Medium Risk", data["summary"]["medium_risk"])
col4.metric("High Risk", data["summary"]["high_risk"])

rows = [{
    "Session": s["session_id"],
    "Protocol": s["protocol"],
    "TLS": s["tls"]["version"] or "N/A",
    "Risk": s["risk"]["level"],
} for s in data["sessions"]]
df = pd.DataFrame(rows)
st.dataframe(df, use_container_width=True)

session_ids = [s["session_id"] for s in data["sessions"]]
selected_id = st.selectbox("Inspect a session", session_ids)
session = next(s for s in data["sessions"] if s["session_id"] == selected_id)

st.subheader(f"Details — {selected_id}")
st.json(session)

```

Run with streamlit run dashboard/app.py. Deliberately bare — a working st.json detail view is the Day 1 bar. Prettify only if Day 2 has slack.

End-of-day milestone: dashboard runs against all three mock files, dropdown switches cleanly, summary cards and table update correctly.

5.4 Day 1 Sync Point (End of Day, 15 Minutes)

Person A shows their real `analysis_result.json` for `good.pcap`. Person B swaps it into the dropdown alongside the mocks. If the dashboard renders it with zero code changes, you're on schedule. If it doesn't, that's a schema mismatch — fix it tonight, not tomorrow morning.

6. Day 2

6.1 Person A — Finish the Pipeline

- Run the same extraction on `weak.pcap` (and `critical.pcap` if sourced). You should see visibly different output — older TLS version, weak cipher, or no TLS at all — versus `good.pcap`. If the outputs look the same, the extraction or the sample data is wrong; check before moving on.
- Attempt STARTTLS detection (`starttls_analyzer.py`) **only after the above is solid**. Method: on the plaintext portion of a session, look for the client sending the `STARTTLS` command and the server's positive response, then check whether subsequent bytes on that same `tcp.stream` become TLS records or stay plaintext. If this isn't reliable by early afternoon, set `starttls.note = "Not available in this capture"` and stop — this is a stretch goal, not a milestone.
- Handle every missing field as `null`, never as an empty string or a fabricated default.
- Produce final `sample_results/good.json`, `weak.json`, `critical.json` from real PCAPs. These replace the mocks entirely.

6.2 Person B — Integrate and Polish

- Swap the mock files for Person A's real output. Point the sidebar dropdown at a glob of `sample_results/*.json` rather than a hardcoded list, so new files from Person A appear automatically.
- Add a PCAP upload widget that calls Person A's `analyzer.py` as a subprocess and writes a fresh JSON — **but keep a button that loads the precomputed `good.json` / `weak.json` directly, bypassing the subprocess entirely**. If live analysis breaks mid-demo, you switch to this without missing a beat.
- Polish: color-code risk levels (red / orange / green) in the table, surface the evidence and recommendation text clearly in the detail view.
- Rehearse the demo (Section 8) at least twice, out loud, with a timer.

7. Fallback Rules — Apply Automatically, Do Not Debate Under Time Pressure

If this happens	Do this
A field can't be reliably extracted	Set it <code>null</code> in the JSON; dashboard simply doesn't render it
STARTTLS detection isn't working	<code>starttls.note = "Not available in this capture"</code> , move on
Live subprocess integration is flaky by Day 2 evening	Demo runs entirely off precomputed JSON; analyzer is shown separately, running from the command line, as proof it's real
tshark field names don't match what's documented here	Run <code>tshark -G fields \ grep <keyword></code> and <code>-v</code> full decode to find the real names for your version
Outputs for <code>good.pcap</code> and <code>weak.pcap</code> look identical	Stop — this means extraction or test data is wrong, not that the servers are equally secure

8. Demo Script (2-3 Minutes)

1. Open dashboard, select `good.json` → show LOW risk summary.
2. Select `weak.json` → show HIGH risk summary, visibly different numbers.
3. Click into the high-risk session → show TLS version, cipher, certificate status, and the specific finding with its severity.
4. Close with one line: *“This is a 2-day proof of concept for a 2-person subset of the full 6-person SIH proposal — the full system adds AI-assisted risk scoring, STARTTLS-stripping detection, and MTA-STS/DANE policy checks on top of this same pipeline.”*

Success bar: two PCAPs in, two different and correct-looking risk outputs out, one real session’s findings inspectable in detail. That’s the whole bar — resist adding anything past it.

9. Judge Q&A — Rehearse These Answers

- **“Isn’t this just Wireshark with extra steps?”** Wireshark decodes the raw handshake — that’s the building block we use — but it has zero judgment about what’s secure, zero grading, zero report. We’re the layer that turns a decode into a security verdict.
- **“How is this different from an active scanner like testssl.sh?”** Those connect to a live server right now. We passively analyze already-captured traffic — useful for forensics, and usable without live access or credentials to the server being investigated.
- **“Why not decrypt and check the actual emails?”** Out of scope by design — we assess the cryptographic health of the connection, not the content, and we don’t have (or want) private keys.
- **“What happens with a TLS extension your tool doesn’t recognize?”** tshark’s dissector is mature and actively maintained; anything genuinely unrecognized is reported as `null` /unknown, never guessed.
- **“How do you know your risk scores are correct?”** Validated against real, known-composition sample captures (clean vs. weak/old TLS) where the ground truth is externally verifiable, not something we invented.
- **“What would you add with more time?”** AI-assisted risk scoring layered on top of the rule-based grades, full STARTTLS-stripping detection, full certificate chain validation against a trust store, and MTA-STS/DANE policy checks.

10. Definition of Done

- ☐ JSON schema agreed and unchanged since Day 1 morning
- ☐ `good.pcap` and `weak.pcap` (real, sourced or captured) produce visibly different, correct-looking `analysis_result.json` output
- ☐ Every unavailable field is `null`, none are fabricated
- ☐ Dashboard renders both real JSON files without any code changes between them
- ☐ Precomputed-JSON fallback path works even with the live subprocess path disabled
- ☐ Demo rehearsed twice, under 3 minutes, both people know the judge Q&A answers
- ☐ `main` branch tagged and pushed, both of you can run it from a clean clone